

Informe de la Etapa II Análisis Sintáctico

Ricardo García, 20-10274
Daniela Gragirena 19-10543
Daniel Quijada 20-10518

1. Formulación

En esta fase del proyecto se nos pide implementar un analizador sintáctico, de programas escritos en BOT. La herramienta que usamos para hacer el análisis sintáctico fue CUP, que es básicamente el analizador sintáctico de Java. La idea es generar un árbol sintáctico abstracto (ASA).

Este programa básicamente, lee un archivo `.bot` y, de ser correcto el código, devuelve el ASA, en caso contrario retorna el primer error sintáctico y todos los errores léxicos. Por temas de simplicidad (y recomendación del profesor), la gramática empleada es ambigua y con recursión izquierda.

2. Implementación

Se utilizó el lenguaje de programación Java, con el analizador léxico `Jflex` y el analizador sintáctico CUP. A grandes rasgos, el código se ejecuta escribiendo en la terminal `./SintBot <Archivo>`. El funcionamiento del código, explicado archivo por archivo es:

- `Main.java`: Es el código principal. Básicamente el programa recibe como argumento la ruta de un archivo `.bot`, luego inicializa los archivos `LexerCup` (el analizador léxico) y `Parser` (el analizador sintáctico). Luego, usando el método `.parse()`, retornamos el nodo raíz del ASA. Ahora si ese nodo llegara a ser `null` el programa retorna una excepción (si no existe el ASA, entonces la frase no es sintéticamente correcta bajo las reglas del lenguaje BOT).
- `Parser.cup` Es el analizador sintáctico y aquí se definen la gramática libre de contexto (GLC) que da la estructura de nuestro lenguaje. También establece la precedencia y asociatividad de los operadores (aritméticos, lógicos y relacionales). Por cada regla gramatical resuelta con éxito, crea un nodo del AST.

- **AST.java** Clase base abstracta. Establece el método `imprimir(int nivel)` que todos los nodos deben implementar para mostrarse recursivamente con tabuladores de indentación.
- **Programa.java** Representa el nodo raíz. Almacena las declaraciones de los robots y el bloque principal de ejecución.
- **Condicion.java** Modela las condiciones que disparan los comportamientos del robot:
 - `activation` (cuando el robot es activado).
 - `deactivation` (cuando es desactivación).
 - `default` (comportamiento por defecto).
- **Instruccion.java** Modela las diferentes sentencias que ejecutan el controlador principal o los robots.
- **Expresion.java** Modela todo tipo de valores y operaciones del lenguaje (binarias, unarias, identificadores)

Ademas, se implemento un archivo `makefile`, que basicamente, ayuda a compilar y ejecutar el codigo solo escribiendo la palabra `make` en la terminal (ojo esto es para sistemas Linux o tipo Unix).

3. Fragmentos de código

```

1 public abstract class AST {
2
3     public abstract void imprimir(int nivel);
4
5     protected void imprimirIndentacion(int nivel) {
6         for (int i = 0; i < nivel; i++) {
7             System.out.print("\t");
8         }
9     }
10 }

```

Listing 1: Contenido de AST.java

```

1 import java.io.*;
2 import java.nio.file.Files;
3 import java.nio.file.Path;
4
5 public class Main {
6
7     public static void main(String[] args) {
8
9         if (args.length == 0) {
10             System.out.println("Error: Especifique un archivo .bot
              como argumento.");

```

```

11         return;
12     }
13
14     String archivo = args[0];
15
16     try {
17         String contenido_archivo = Files.readString(Path.of(
18             archivo));
19
20         Reader strRdr = new StringReader(contenido_archivo);
21         LexerCup lexer = new LexerCup(strRdr);
22         Parser parser = new Parser(lexer);
23
24         AST nodoAST = (AST) parser.parse().value;
25
26         if (nodoAST != null) {
27             nodoAST.imprimir(0);
28         }
29     } catch (Exception e) {
30         System.err.println("Ocurrio un error durante la
31         compilacion: " + e.getMessage());
32     }
33 }

```

Listing 2: Contenido de Main.java

```

1 import java.util.ArrayList;
2
3 public class Programa extends AST {
4     private ArrayList<Declaracion> declaraciones;
5     private Instruccion instruccion;
6
7     public Programa(ArrayList<Declaracion> declaraciones, Instruccion
8         instruccion) {
9         this.declaraciones = declaraciones;
10        this.instruccion = instruccion;
11    }
12
13    @Override
14    public void imprimir(int nivel) {
15        imprimirIndentacion(nivel);
16        System.out.println("PROGRAMA BOT");
17
18        if (declaraciones != null && !declaraciones.isEmpty()) {
19            imprimirIndentacion(nivel + 1);
20            System.out.println("DECLARACIONES:");
21            for (Declaracion decl : declaraciones) {
22                decl.imprimir(nivel + 2);
23            }
24        }
25    }
26 }

```

```

24
25     if (instruccion != null) {
26         imprimirIndentacion(nivel + 1);
27         System.out.println("EXECUTE:");
28         instruccion.imprimir(nivel + 2);
29     }
30 }
31 }

```

Listing 3: Contenido de Programa.java

```

- Condicion: default
- ALMACENAMIENTO:
    - literal entero: 35
EXECUTE:
- SECUENCIACION:
    ACTIVACION:
        - var: contador
    - CONDICIONAL:
        EXP_RELACIONAL:
            - operacion: >
            - expresion izquierda:
                - var: bar
            - expresion derecha:
                - literal entero: 2
        AVANCE:
            - var: contador

```

Figura 1: Ejemplo de salida correcta